Hands on 1

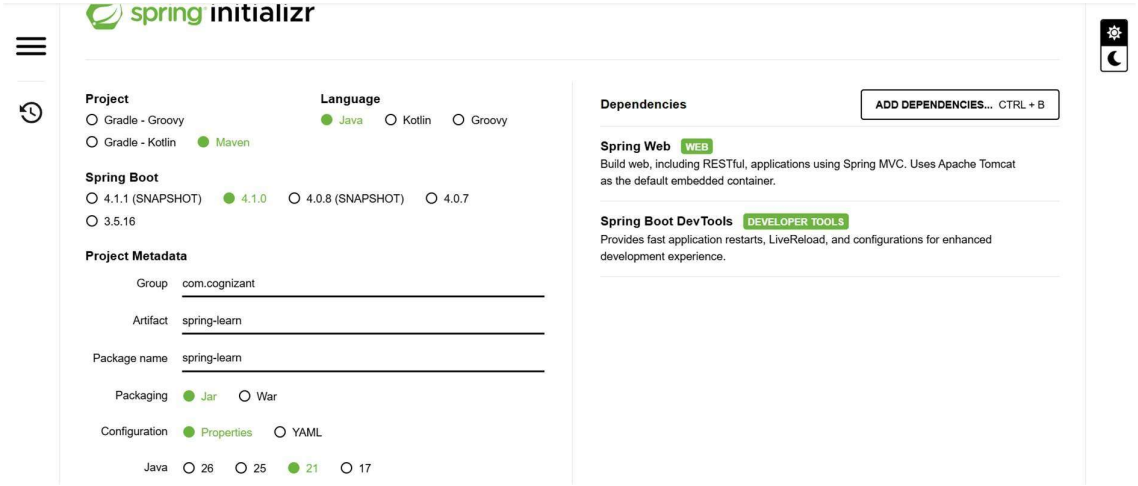
Create a Spring Web Project using Maven

Follow steps below to create a project:

1. Go to https://start.spring.io/
2. Change Group as “com.cognizant”
3. Change Artifact Id as “spring-learn”
4. Select Spring Boot DevTools and Spring Web
5. Create and download the project as zip
6. Extract the zip in root folder to Eclipse Workspace
7. Build the project using ‘mvn clean package -Dhttp.proxyHost=proxy.cognizant.com -Dhttp.proxyPort=6050 Dhttps.proxyHost=proxy.cognizant.com -Dhttps.proxyPort=6050 -Dhttp.proxyUser=123456’ command in command line
8. Import the project in Eclipse "File > Import > Maven > Existing Maven Projects > Click Browse and select extracted folder > Finish"
9. Include logs to verify if main() method of SpringLearnApplication.
10. Run the SpringLearnApplication class.

SME to walk through the following aspects related to the project created:

1. src/main/java - Folder with application code
2. src/main/resources - Folder for application configuration
3. src/test/java - Folder with code for testing the application
4. SpringLearnApplication.java - Walkthrough the main() method.
5. Purpose of @SpringBootApplication annotation
6. pom.xml
    1. Walkthrough all the configuration defined in XML file
    2. Open 'Dependency Hierarchy' and show the dependency tree.



Project Structure src/main/java

Contains the Java source code of the application.

src/main/resources

Contains configuration files such as application.properties.

src/test/java

Contains test classes used for unit testing.

SpringLearnApplication.java

The main class of the Spring Boot application. It contains the main() method, which

starts the application using SpringApplication.run(). pom.xml

The Maven configuration file that contains project information, dependencies, plugins, and build configuration.

Purpose of @SpringBootApplication

@SpringBootApplication is the main annotation used in Spring Boot. It combines:

- @Configuration
- @EnableAutoConfiguration
- @ComponentScan

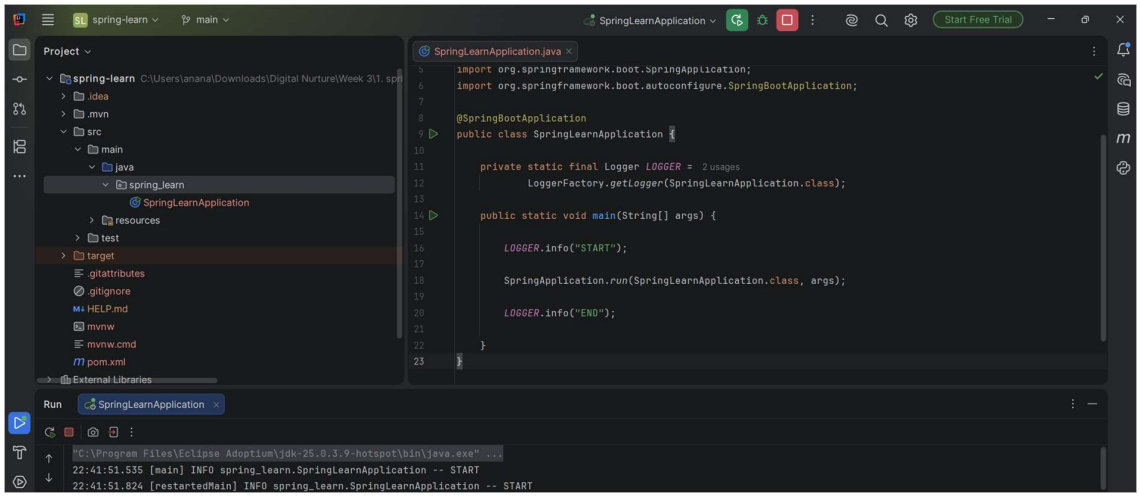
It enables auto-configuration, component scanning, and Java-based configuration for the application.

Logging Code

```
package spring_learn;

import org.slf4j.Logger; import
org.slf4j.LoggerFactory;
import org.springframework.boot.SpringApplication;
import org.springframework.boot.autoconfigure.SpringBootApplication;

@SpringBootApplication
public class SpringLearnApplication {

   private static final Logger LOGGER =
         LoggerFactory.getLogger(SpringLearnApplication.class);

   public static void main(String[] args) {

      LOGGER.info("START");

      SpringApplication.run(SpringLearnApplication.class, args);

      LOGGER.info("END");
   }
}
```

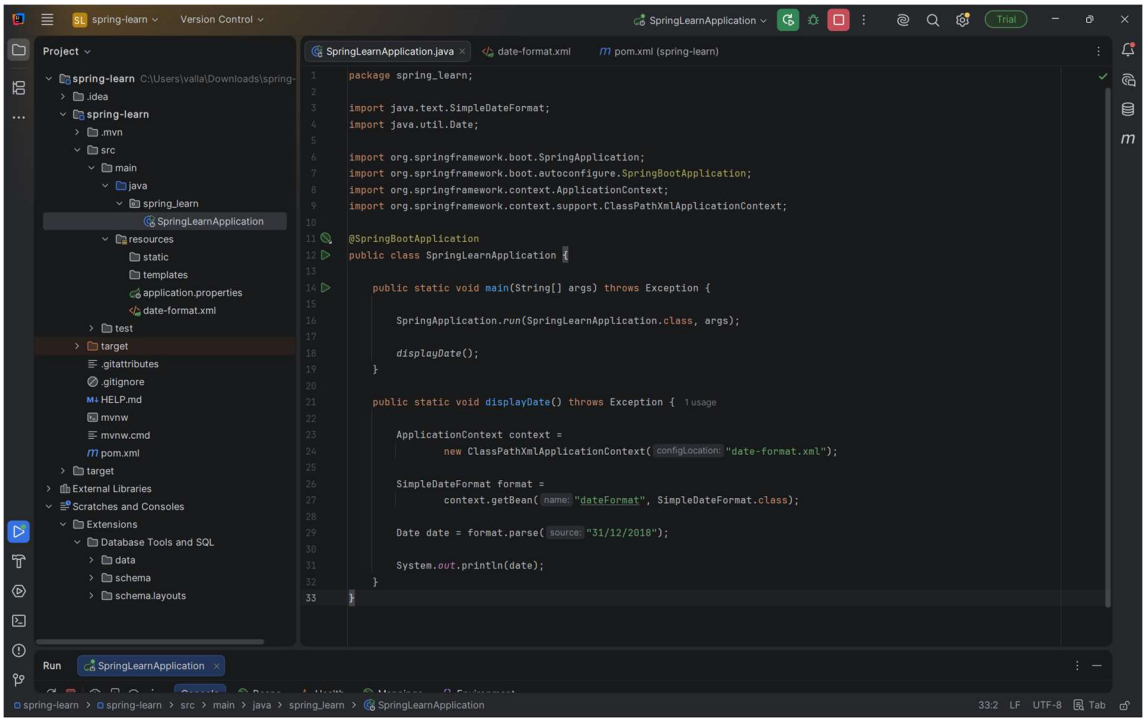
Output

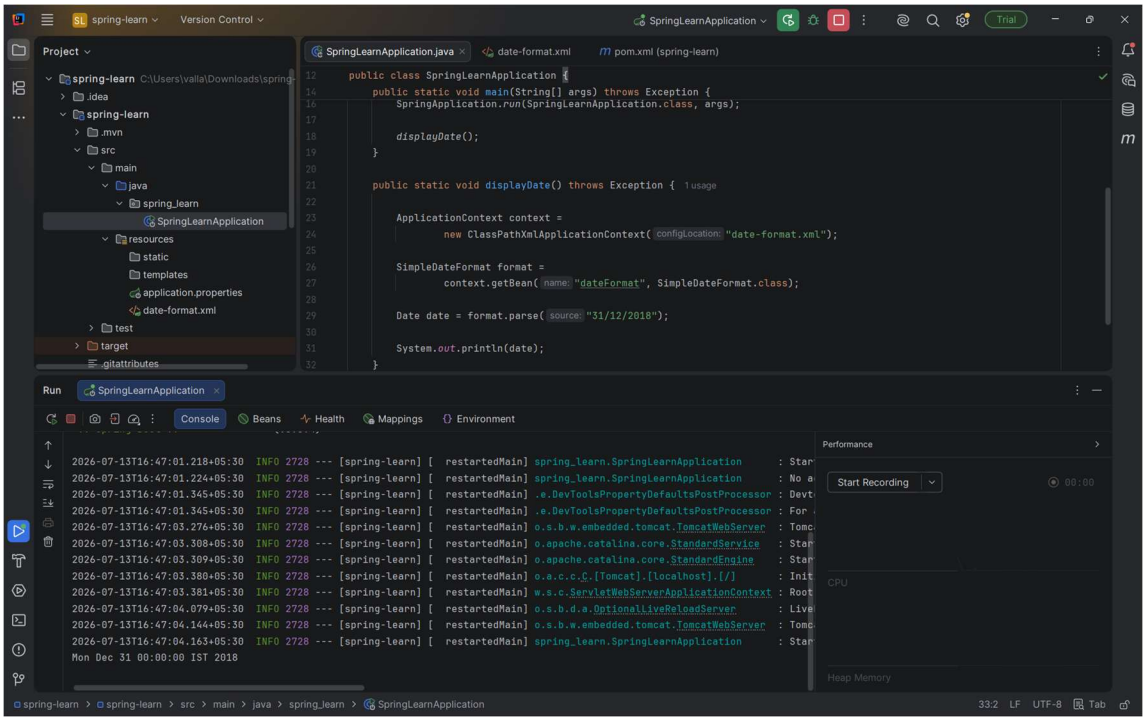
```
INFO SpringLearnApplication -- START

:: Spring Boot ::

Tomcat started on port(s): 8080

Started SpringLearnApplication

INFO SpringLearnApplication -- END
```

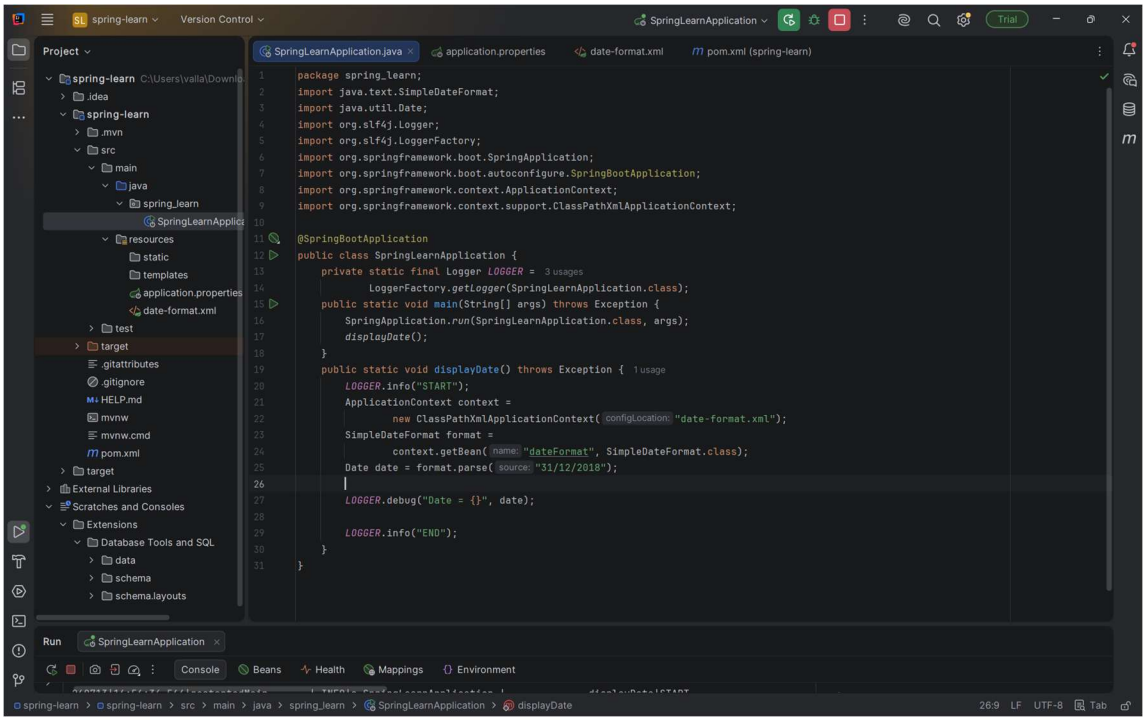


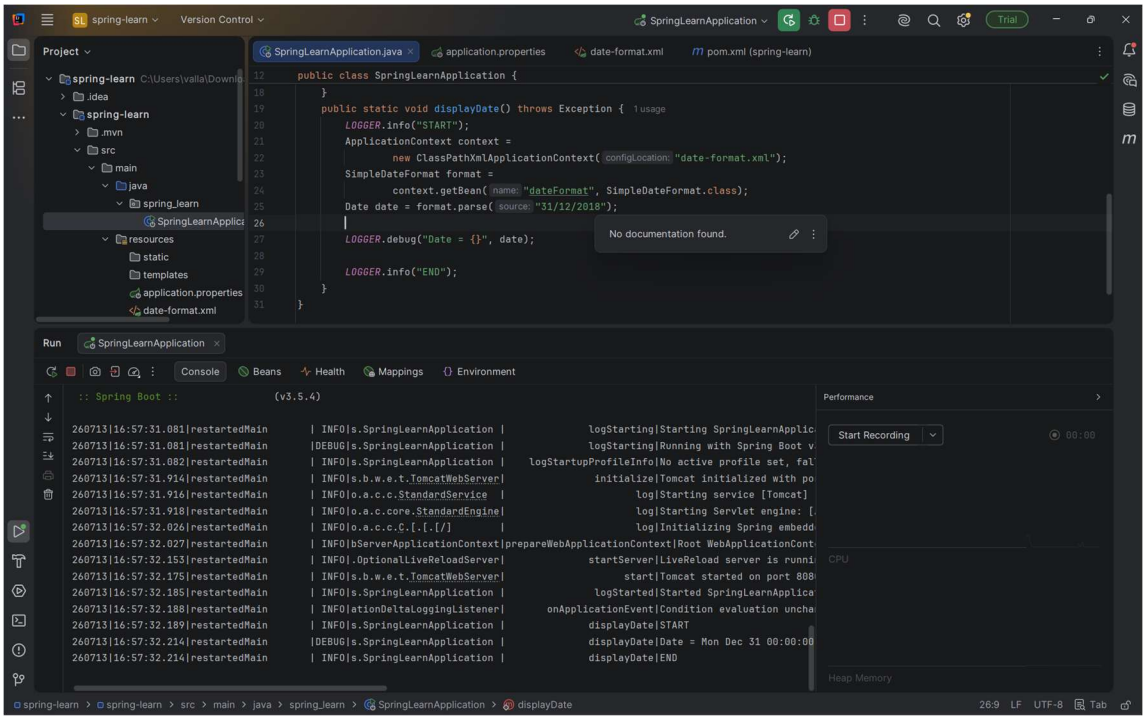
Result

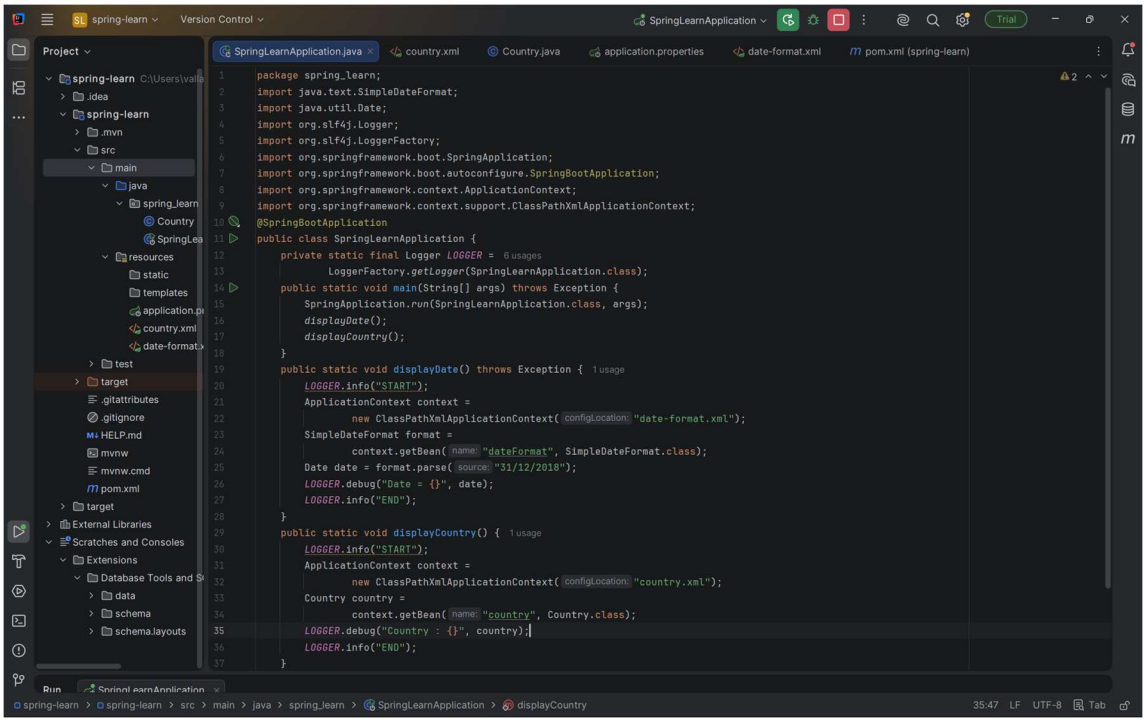
A Spring Boot Web project was successfully created using Maven. The application was executed successfully, and the logging statements verified the execution of the main() method.

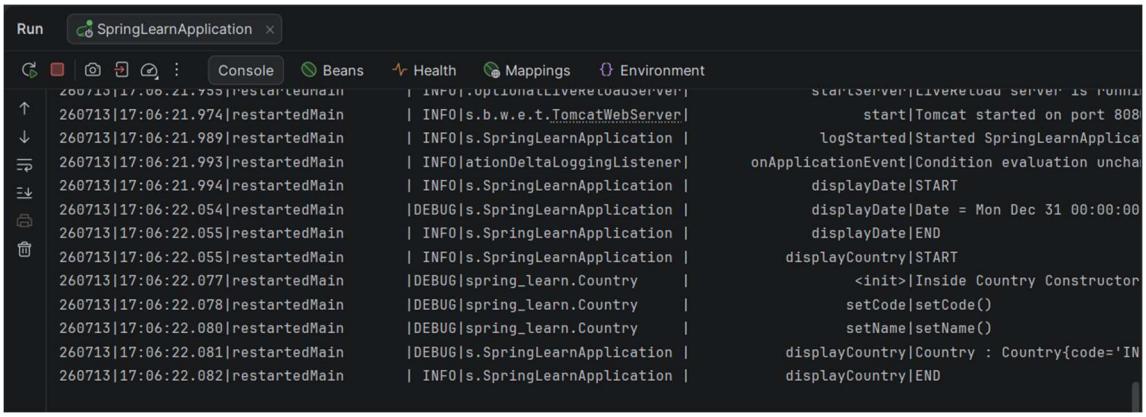

spring-learn
Version Control
SpringLearnApplication
Trial
Project
spring-learn C:\Users\valla\Downloads\spring-
.idea
spring-learn
.mvn
src
main
java
spring_learn
SpringLearnApplication
resources
static
templates
application.properties
date-format.xml
test
target
.gitattributes
.gitignore
HELP.md
mvnw
mvnw.cmd
pom.xml
target
External Libraries
Scratches and Consoles
Extensions
Database Tools and SQL
data
schema
schema.layouts
SpringLearnApplication.java
date-format.xml
pom.xml (spring-learn)
package spring_learn;
import java.text.SimpleDateFormat;
import java.util.Date;
import org.springframework.boot.SpringApplication;
import org.springframework.boot.autoconfigure.SpringBootApplication;
import org.springframework.context.ApplicationContext;
import org.springframework.context.support.ClassPathXmlApplicationContext;
@SpringBootApplication
public class SpringLearnApplication {
public static void main(String[] args) throws Exception {
SpringApplication.run(SpringLearnApplication.class, args);
displayDate();
}
public static void displayDate() throws Exception { 1 usage
ApplicationContext context =
new ClassPathXmlApplicationContext(configLocation: "date-format.xml");
SimpleDateFormat format =
context.getBean(name: "dateFormat", SimpleDateFormat.class);
Date date = format.parse(source: "31/12/2018");
System.out.println(date);
}
}
Run
SpringLearnApplication
spring-learn > spring-learn > src > main > java > spring_learn > SpringLearnApplication
33:2
LF
UTF-8
Tab

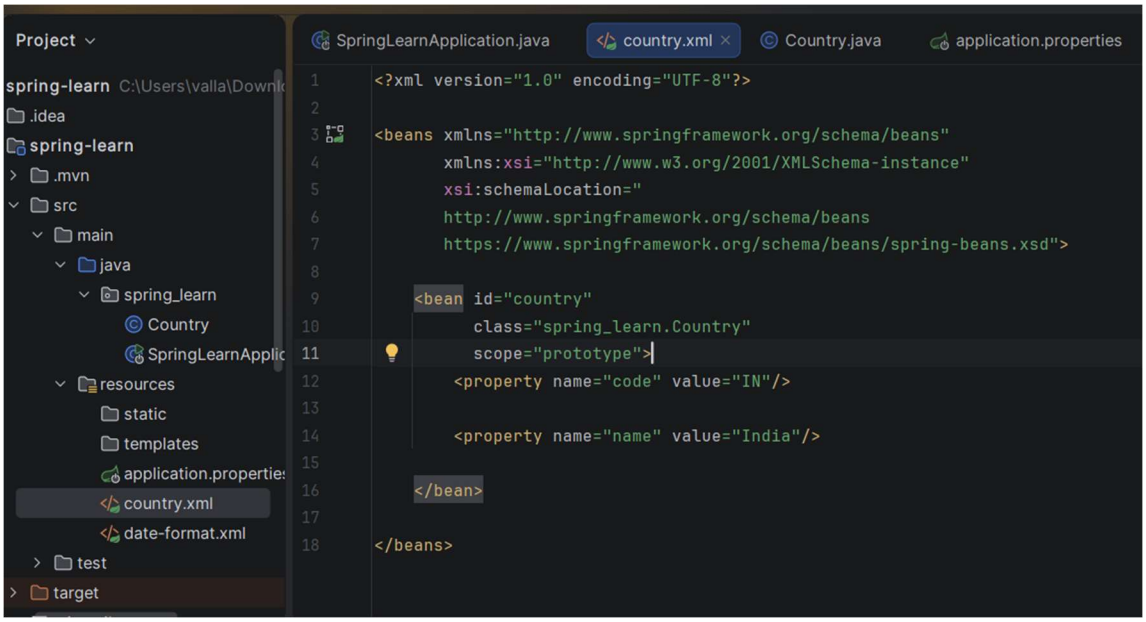

spring-learn
Version Control
SpringLearnApplication
Trial
Project
SpringLearnApplication.java
date-format.xml
pom.xml (spring-learn)
public class SpringLearnApplication {
public static void main(String[] args) throws Exception {
SpringApplication.run(SpringLearnApplication.class, args);
displayDate();
}
public static void displayDate() throws Exception { 1 usage
ApplicationContext context =
new ClassPathXmlApplicationContext(configLocation: "date-format.xml");
SimpleDateFormat format =
context.getBean(name: "dateFormat", SimpleDateFormat.class);
Date date = format.parse(source: "31/12/2018");
System.out.println(date);
}
Run
SpringLearnApplication
Console
Beans
Health
Mappings
Environment
2026-07-13T16:47:01.218+05:30 INFO 2728 --- [spring-learn] [restartedMain] spring_learn.SpringLearnApplication
2026-07-13T16:47:01.224+05:30 INFO 2728 --- [spring-learn] [restartedMain] spring_learn.SpringLearnApplication
2026-07-13T16:47:01.345+05:30 INFO 2728 --- [spring-learn] [restartedMain] .e.DevToolsPropertyDefaultsPostProcessor
2026-07-13T16:47:01.345+05:30 INFO 2728 --- [spring-learn] [restartedMain] .e.DevToolsPropertyDefaultsPostProcessor
2026-07-13T16:47:03.276+05:30 INFO 2728 --- [spring-learn] [restartedMain] o.s.b.w.embedded.tomcat.TomcatWebServer
2026-07-13T16:47:03.308+05:30 INFO 2728 --- [spring-learn] [restartedMain] o.apache.catalina.core.StandardService
2026-07-13T16:47:03.309+05:30 INFO 2728 --- [spring-learn] [restartedMain] o.apache.catalina.core.StandardEngine
2026-07-13T16:47:03.380+05:30 INFO 2728 --- [spring-learn] [restartedMain] o.a.c.c.C.[Tomcat].[localhost].[/]
2026-07-13T16:47:03.381+05:30 INFO 2728 --- [spring-learn] [restartedMain] w.s.c.ServletWebServerApplicationContext
2026-07-13T16:47:04.079+05:30 INFO 2728 --- [spring-learn] [restartedMain] o.s.b.d.a.OptionalLiveReloadServer
2026-07-13T16:47:04.144+05:30 INFO 2728 --- [spring-learn] [restartedMain] o.s.b.w.embedded.tomcat.TomcatWebServer
2026-07-13T16:47:04.163+05:30 INFO 2728 --- [spring-learn] [restartedMain] spring_learn.SpringLearnApplication
Mon Dec 31 00:00:00 IST 2018
Performance
Start Recording
00:00
CPU
Heap Memory
spring-learn > spring-learn > src > main > java > spring_learn > SpringLearnApplication
33:2
LF
UTF-8
Tab

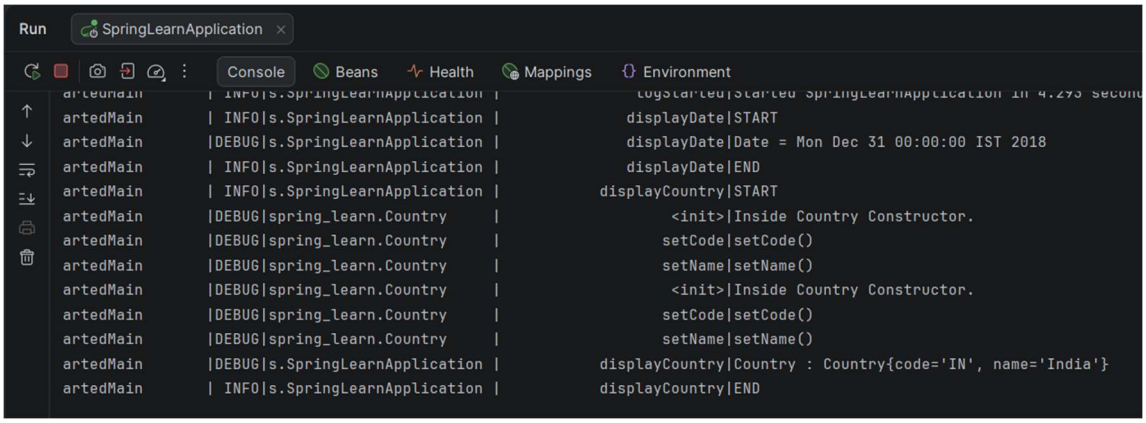

spring-learn
Version Control
SpringLearnApplication
Trial
Project
SpringLearnApplication.java
application.properties
date-format.xml
pom.xml (spring-learn)
spring-learn C:\Users\valla\Downlo
.idea
spring-learn
.mvn
src
main
java
spring_learn
SpringLearnApplica
resources
static
templates
application.properties
date-format.xml
test
target
.gitattributes
.gitignore
HELP.md
mvnw
mvnw.cmd
pom.xml
target
External Libraries
Scratches and Consoles
Extensions
Database Tools and SQL
data
schema
schema.layouts
package spring_learn;
import java.text.SimpleDateFormat;
import java.util.Date;
import org.slf4j.Logger;
import org.slf4j.LoggerFactory;
import org.springframework.boot.SpringApplication;
import org.springframework.boot.autoconfigure.SpringBootApplication;
import org.springframework.context.ApplicationContext;
import org.springframework.context.support.ClassPathXmlApplicationContext;
@SpringBootApplication
public class SpringLearnApplication {
private static final Logger LOGGER = 3 usages
LoggerFactory.getLogger(SpringLearnApplication.class);
public static void main(String[] args) throws Exception {
SpringApplication.run(SpringLearnApplication.class, args);
displayDate();
}
public static void displayDate() throws Exception { 1 usage
LOGGER.info("START");
ApplicationContext context =
new ClassPathXmlApplicationContext(configLocation: "date-format.xml");
SimpleDateFormat format =
context.getBean(name: "dateFormat", SimpleDateFormat.class);
Date date = format.parse(source: "31/12/2018");
LOGGER.debug("Date = {}", date);
LOGGER.info("END");
}
}
Run
SpringLearnApplication
Console
Beans
Health
Mappings
Environment
spring-learn > spring-learn > src > main > java > spring_learn > SpringLearnApplication > displayDate
26:9
LF
UTF-8
Tab


spring-learn
Version Control
SpringLearnApplication
Trial
Project
SpringLearnApplication.java
application.properties
date-format.xml
pom.xml (spring-learn)
public class SpringLearnApplication {
}
public static void displayDate() throws Exception { 1 usage
LOGGER.info("START");
ApplicationContext context =
new ClassPathXmlApplicationContext(configLocation: "date-format.xml");
SimpleDateFormat format =
context.getBean(name: "dateFormat", SimpleDateFormat.class);
Date date = format.parse(source: "31/12/2018");
LOGGER.debug("Date = {}", date);
LOGGER.info("END");
}
}
No documentation found.
Run
SpringLearnApplication
Console
Beans
Health
Mappings
Environment
:: Spring Boot :: (v3.5.4)
260713|16:57:31.081|restartedMain | INFO|s.SpringLearnApplication | logStarting|Starting SpringLearnApplic
260713|16:57:31.081|restartedMain |DEBUG|s.SpringLearnApplication | logStarting|Running with Spring Boot v
260713|16:57:31.082|restartedMain | INFO|s.SpringLearnApplication | logStartupProfileInfo|No active profile set, fal
260713|16:57:31.914|restartedMain | INFO|s.b.w.e.t.TomcatWebServer| initialize|Tomcat initialized with po
260713|16:57:31.916|restartedMain | INFO|o.a.c.c.StandardService | log|Starting service [Tomcat]
260713|16:57:31.918|restartedMain | INFO|o.a.c.c.core.StandardEngine| log|Starting Servlet engine: [
260713|16:57:32.026|restartedMain | INFO|o.a.c.c.C.[.[.[/] | log|Initializing Spring embedd
260713|16:57:32.027|restartedMain | INFO|bServerApplicationContext|prepareWebApplicationContext|Root WebApplicationCont
260713|16:57:32.153|restartedMain | INFO|.OptionalLiveReloadServer| startServer|LiveReload server is runni
260713|16:57:32.175|restartedMain | INFO|s.b.w.e.t.TomcatWebServer| start|Tomcat started on port 808
260713|16:57:32.185|restartedMain | INFO|s.SpringLearnApplication | logStarted|Started SpringLearnApplica
260713|16:57:32.188|restartedMain | INFO|ationDeltaLoggingListener| onApplicationEvent|Condition evaluation uncha
260713|16:57:32.189|restartedMain | INFO|s.SpringLearnApplication | displayDate|START
260713|16:57:32.214|restartedMain |DEBUG|s.SpringLearnApplication | displayDate|Date = Mon Dec 31 00:00:00
260713|16:57:32.214|restartedMain | INFO|s.SpringLearnApplication | displayDate|END
Performance
Start Recording
00:00
CPU
Heap Memory
spring-learn > spring-learn > src > main > java > spring_learn > SpringLearnApplication > displayDate
26:9
LF
UTF-8
Tab


spring-learn
Version Control
SpringLearnApplication
Trial
Project
SpringLearnApplication.java
country.xml
Country.java
application.properties
date-format.xml
pom.xml (spring-learn)
spring-learn C:\Users\valla
.idea
spring-learn
.mvn
src
main
java
spring_learn
Country
SpringLea
resources
static
templates
application.p
country.xml
date-format
test
target
.gitattributes
.gitignore
HELP.md
mvnw
mvnw.cmd
pom.xml
target
External Libraries
Scratches and Consoles
Extensions
Database Tools and S
data
schema
schema.layouts
package spring_learn;
import java.text.SimpleDateFormat;
import java.util.Date;
import org.slf4j.Logger;
import org.slf4j.LoggerFactory;
import org.springframework.boot.SpringApplication;
import org.springframework.boot.autoconfigure.SpringBootApplication;
import org.springframework.context.ApplicationContext;
import org.springframework.context.support.ClassPathXmlApplicationContext;
@SpringBootApplication
public class SpringLearnApplication {
private static final Logger LOGGER = 6 usages
LoggerFactory.getLogger(SpringLearnApplication.class);
public static void main(String[] args) throws Exception {
SpringApplication.run(SpringLearnApplication.class, args);
displayDate();
displayCountry();
}
public static void displayDate() throws Exception { 1 usage
LOGGER.info("START");
ApplicationContext context =
new ClassPathXmlApplicationContext(configLocation: "date-format.xml");
SimpleDateFormat format =
context.getBean(name: "dateFormat", SimpleDateFormat.class);
Date date = format.parse(source: "31/12/2018");
LOGGER.debug("Date = {}", date);
LOGGER.info("END");
}
public static void displayCountry() { 1 usage
LOGGER.info("START");
ApplicationContext context =
new ClassPathXmlApplicationContext(configLocation: "country.xml");
Country country =
context.getBean(name: "country", Country.class);
LOGGER.debug("Country : {}", country);
LOGGER.info("END");
}
Run
spring-learn > spring-learn > src > main > java > spring_learn > SpringLearnApplication > displayCountry
35:47
LF
UTF-8
Tab


Run
SpringLearnApplication
Console
Beans
Health
Mappings
Environment
260713|17:06:21.974|restartedMain | INFO|s.b.w.e.t.TomcatWebServer| start|Tomcat started on port 808
260713|17:06:21.989|restartedMain | INFO|s.SpringLearnApplication | logStarted|Started SpringLearnApplica
260713|17:06:21.993|restartedMain | INFO|ationDeltaLoggingListener| onApplicationEvent|Condition evaluation uncha
260713|17:06:21.994|restartedMain | INFO|s.SpringLearnApplication | displayDate|START
260713|17:06:22.054|restartedMain |DEBUG|s.SpringLearnApplication | displayDate|Date = Mon Dec 31 00:00:00
260713|17:06:22.055|restartedMain | INFO|s.SpringLearnApplication | displayDate|END
260713|17:06:22.055|restartedMain | INFO|s.SpringLearnApplication | displayCountry|START
260713|17:06:22.077|restartedMain |DEBUG|spring_learn.Country | <init>|Inside Country Constructor
260713|17:06:22.078|restartedMain |DEBUG|spring_learn.Country | setCode|setCode()
260713|17:06:22.080|restartedMain |DEBUG|spring_learn.Country | setName|setName()
260713|17:06:22.081|restartedMain |DEBUG|s.SpringLearnApplication | displayCountry|Country : Country{code='IN
260713|17:06:22.082|restartedMain | INFO|s.SpringLearnApplication | displayCountry|END





Hands on 4

Spring Core – Load Country from Spring Configuration XML

An airlines website is going to support booking on four countries. There will be a drop down on the home page of this website to select the respective country. It is also important to store the two-character ISO code of each country.

Code	Name
US	United States
DE	Germany
IN	India

JP	Japan

Above data has to be stored in spring configuration file. Write a program to read this configuration file and display the details.

Steps to implement

- Pick any one of your choice country to configure in Spring XML configuration named country.xml.
- Create a bean tag in spring configuration for country and set the property and values

```
<bean id="country" class="com.cognizant.springlearn.Country">
  <property name="code" value="IN" />
  <property name="name" value="India" />
</bean>
```

- Create Country class with following aspects:
  - Instance variables for code and name
  - Implement empty parameter constructor with inclusion of debug log within the constructor with log message as “Inside Country Constructor.”
  - Generate getters and setters with inclusion of debug with relevant message within each setter and getter method. o Generate toString() method
- Create a method displayCountry() in SpringLearnApplication.java, which will read the country bean from spring configuration file and display the country details. ClassPathXmlApplicationContext, ApplicationContext and context.g etBean(“beanId”, Country.class). Refer sample code for displayCountry() method below.

```
ApplicationContext context = new
ClassPathXmlApplicationContext("country.xml");

Country country = (Country) context.getBean("country", Country.class);

LOGGER.debug("Country : {}", country.toString());
```

- Invoke displayCountry() method in main() method of SpringLearnApplication.java.
- Execute main() method and check the logs to find out which constructors and methods were invoked.

SME to provide more detailing about the following aspects:

- bean tag, id attribute, class attribute, property tag, name attribute, value attribute
- ApplicationContext, ClassPathXmlApplicationContext
- What exactly happens when context.getBean() is invoked

Aim

To load a Country bean from a Spring XML configuration file using the Spring IoC container and display the country details.

Procedure

1. Create a Country class with code and name properties.
2. Add an empty constructor with a debug log.
3. Generate getters and setters with debug logs.
4. Override the toString() method.
5. Create country.xml in the resources folder.
6. Configure the country bean with code and name properties.
7. Read the bean using ClassPathXmlApplicationContext.
8. Display the country details using LOGGER.debug().
9. Run the application and verify the logs.

Files Created

Country.java

- Contains instance variables:
    - code o name
- Includes: o Default constructor o Getters o Setters o toString()

---

country.xml

```
<bean id="country" class="spring_learn.Country">
   <property name="code" value="IN"/>
   <property name="name" value="India"/>
```

```
</bean>
```

SpringLearnApplication.java Uses:

```
ApplicationContext context =
        new ClassPathXmlApplicationContext("country.xml");

Country country =
        context.getBean("country", Country.class);

LOGGER.debug("Country : {}", country.toString());
```

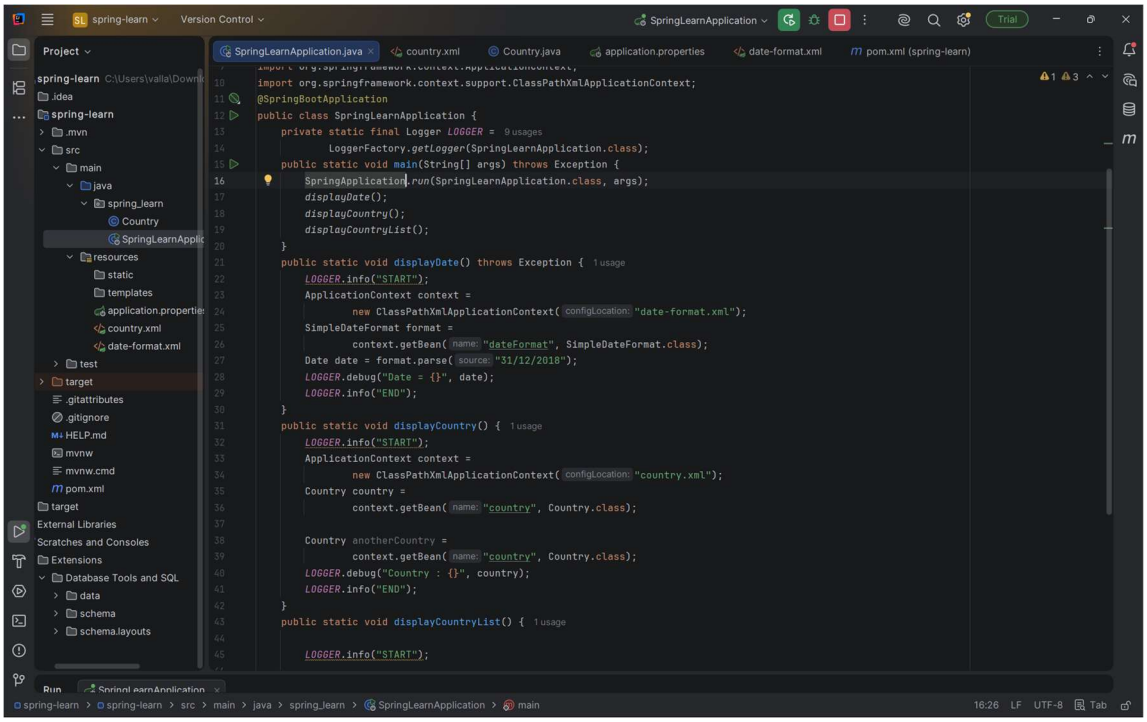
Output

```
Inside Country Constructor.

Inside setCode()

Inside setName()

Country : Country [code=IN, name=India]
```

(Attach your console screenshot if required.)

Result

The Country bean was successfully loaded from the Spring XML configuration file using ClassPathXmlApplicationContext, and the country details were displayed successfully.




```
    public static void displayCountryList() {  1 usage

        LOGGER.info("START");

        ApplicationContext context =
                new ClassPathXmlApplicationContext( configLocation: "country.xml");

        List<?> countryList =
                context.getBean( name: "countryList", List.class);

        LOGGER.debug("Country List : {}", countryList);

        LOGGER.info("END");
    }
}
```

```
                setName|setName()
    displayCountryList|Country List : [Country{code='IN', name='India'}, Country{code='US',
    displayCountryList|END
```

```
, name='United States'}, Country{code='DE', name='Germany'}, Country{code='JP', name='Japan'}]
```